UNIVERSITÄT TÜBINGEN
Prof. Dr.-Ing. Hendrik P.A. Lensch
Lehrstuhl für Computergrafik
Philipp Langsteiner (philipp.langsteiner@uni-tuebingen.de)
Mehran Hosseinzadeh (mehran.hosseinzadeh@uni-tuebingen.de)
Deborah Kornwolf (deborah.kornwolf@student.uni-tuebingen.de)

EBERHARD KARLS
UNIVERSITÄT
TÜBINGEN

March 10, 2026

Massively Parallel Computing
Assignment 7

Instructions

Present your results to the exercise instructors to get a grading on this exercise sheet. You can earn two **BEE** by presenting all exercises on this sheet.

7.1 NBody Simulation (100 P)

- Start with the source code in the `NBody` folder.
- You will have to link against libGL.so (`-lGL` flag) and libX11.so (`-lX11` flag) to build the program. When using CMake, this is already configured for you. In Eclipse, add them as shown in Figure **??**.
- The provided template already contains the memory management, visualization setup, and the helper code for the grid data structure. Your task in this exercise is to complete the missing CUDA kernels for the actual simulation and to launch them correctly from `main.cu`. In particular, the missing work is concentrated in the acceleration kernels, the integration kernel, and the corresponding kernel launches inside the simulation loop.
- Implement the following ToDos:

  1. **Part 1 (Bruteforce):** Write a kernel that updates the accelerations of all bodies based on the gravitational attraction of all other bodies once per frame. This is the kernel which does most of the work.
     The acceleration contribution of body j to body i is

     $$a_{ij} = \frac{m_j r_{ij}}{(\|r_{ij}\|^2 + \varepsilon^2)^{3/2}} \tag{1}$$

     and the total acceleration is obtained by summing these contributions over all bodies:

     $$a_i = \sum_{j=0}^{N-1} a_{ij} \tag{2}$$

     where
     - m_j is the mass of body j,
     - r_{ij} is the vector from body i to body j and
     - ε^2 is a damping factor given as `EPS_2` in the code.

     In the provided code, the factor `GRAVITY` is applied during the integration step when updating the velocity.
  2. Launch the bruteforce kernel in the simulation loop so that it is executed once per frame when the program runs in bruteforce mode.
  3. Write a kernel that updates the velocities and positions of all bodies based on their accelerations once per frame.

- To update a body's velocity, add the acceleration scaled by `GRAVITY` to its velocity.
- To update a body's position, add the updated velocity to the position.

4. Launch the integration kernel once per frame after the accelerations have been computed.
5. **Part 2 (Grid-Based Approximation):** Implement the acceleration kernel for the provided grid-based data structure:
   - For nearby bodies, compute interactions exactly by looking at bodies in the body's own cell and the relevant neighboring grid cells.
   - For far-away regions, use the provided coarse-grid center-of-mass approximation instead of evaluating all distant bodies individually.
   - For a coarse cell with total mass $M = \sum_k m_k$, its center of mass is

$$c = \frac{1}{M} \sum_k m_k p_k \tag{3}$$

     and far-away coarse cells should be approximated as one body with mass M at position c.
   - Store the resulting acceleration for each body in the acceleration buffer.
6. Launch the kernels needed for the grid-based mode in the simulation loop so that the intermediate data structures for cell assignment, grouping, cell ranges, and the coarse-grid approximation are prepared before the grid-based acceleration kernel is executed.
7. Compare runtime and visual behavior of the grid-based mode against the bruteforce version.

- *Hints:*
  - Calculate the acceleration of one body per thread.
  - Start with the bruteforce implementation first and verify correctness visually before working on the grid-based version.
  - Read the provided helper functions and data structures carefully before implementing the missing kernels and launches.
  - In the grid-based mode, think about which intermediate data has to exist before accelerations can be computed.
  - For the grid-based mode, it is useful to think of the workflow in four stages: assign each body to a cell, group/sort bodies by cell, build the cell ranges, then compute the coarse-grid approximation before launching the acceleration kernel.
  - You can find detailed information about GPU N-body optimization in https://developer.nvidia.com/gpugems/GPUGems3/gpugems3_ch31.html.
  - Reduce `NUM_FRAMES` to 100 while profiling - not lower because the GPU's boost clock might not be reached otherwise.

7.2 Implement your own Idea

Design and implement your own massively parallel CUDA project to earn a second BEE. Choose an idea that clearly benefits from GPU parallelism, for example hydraulic or thermal erosion, a fluid or smoke simulation, reaction-diffusion, a particle effect, or another project of comparable scope. Present your implementation together with a short explanation of the idea, the parallelization strategy, and the observed results.